

RESEARCH METHODS

Boosting multiplexing capabilities for error-robust spatial transcriptomic methods using a set exchange approach

Johan Boström^{1*}, Michał Zapała², Igor Adameyko^{1,3*}

In the last decades, image-based transcriptomic and proteomic experiments have moved from single-target probes to multiplexed experiments, allowing researchers to study hundreds or even thousands of mRNA and protein targets simultaneously. This large increase in scope necessitates methods in either increased specificity or in error correction, such as the Hamming codes used in the imaging-based spatial transcriptomic method MERFISH. For some experimental conditions, Hamming codes are efficient in encoding the highest possible number of genes for spatial analysis. However, for most experimental parameters, the optimal generation of error-robust codebooks is an unsolved mathematical problem. Here, we present a method to generate highly optimized extended Hamming codebooks compatible with established error-correctable methodologies such as MERFISH. Our method uses an iterative set-exchange approach and generally reaches over 90% of the theoretical maximum limit of gene set complexity. We also provide ready-to-use codebooks and discuss the advantages and disadvantages of changing probe density.

INTRODUCTION

Traditionally, molecular biology has used one laboratory reagent for one target. Researchers often become experts in evaluating what is a proper signal and what is noise, usually with nuances for specific antibodies or nucleic acid probes. With recent advantages in spatial transcriptomics, multiplexing approaches have led to each captured image no longer representing a known target, but the full combination of images being decodable into information about mRNA molecules (1). Thus, the decisions of what is signal and what is noise are becoming fully computerized. As anyone who has ever seen an image of a fluorescent target can attest, noise is ever-present, and the signal intensity can vary due to laboratory, optical, or tissue-specific issues. All of these introduce errors and thus necessitate either error-elimination or error-handling processes.

Error-elimination methodologies overcome this hurdle by increasing the signal-noise ratio, for example, with padlock probes and rolling amplification providing the effective detection of gene expression in a slice of a tissue. Other methods with powerful multiplexing capacity, such as MERFISH (multiplexed error-robust fluorescence in situ hybridization), accept that errors will inevitably happen and use error-handling code strategies to identify and deal with them (2, 3). Error correction is a field developed by necessity in the 1950s to solve issues with computerized reading of punch cards (4), where one misread bit could destroy an entire session. The application of error correction to MERFISH is achieved by assigning a barcode to each gene, a binary sequence, and each barcode is different in four different positions to every other used barcode. If a single error happens, the erroneous information is still sufficient to trace back to the correct barcode, and if two errors happens simultaneously, it can be identified as a double-bit error. This level of error robustness is called single error correction, double error detection

(SECDED) and is also used in modern computer hardware storage. For biological purposes, this is useful when multiple mRNA species need to be detected in a highly multiplexed fashion within biological tissue, under suboptimal experimental conditions (3).

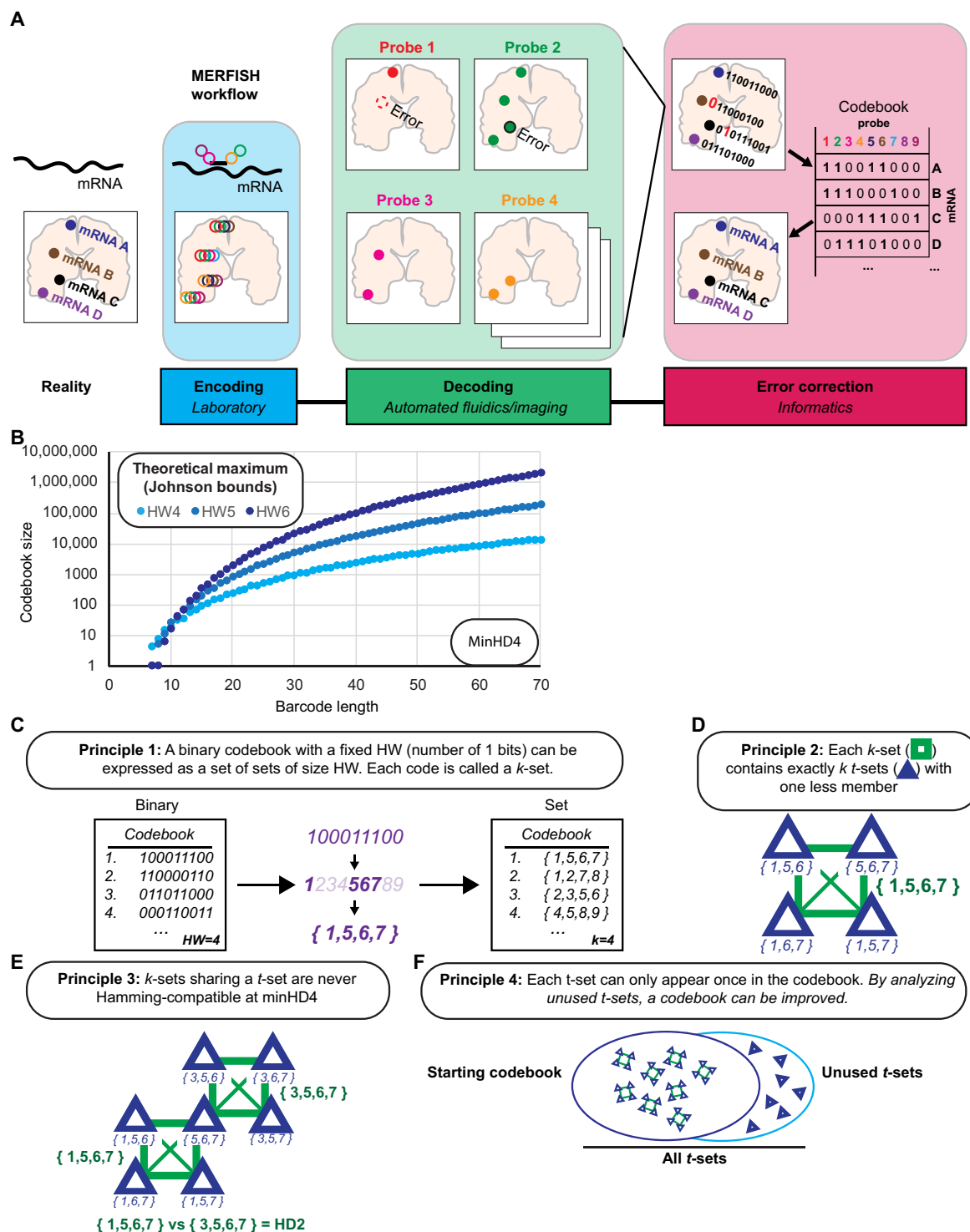
Although challenging, the current progress in multiplexed single-cell methods keeps transforming modern biology and biomedicine. Most recently, a plethora of single-cell spatial methods resulted in a surge of discoveries linking the positions of cells to diversity of cell phenotypes, as recovered at transcriptional or epigenetic levels (5–7). Such addition of a spatial tissue context has delivered an understanding of local and distant signals defining the development, regeneration, healthy self-renewal of organs, or pathological processes such as cancer (1, 8). Thus, spatial technologies operating at single-cell resolution has become one of the most stellar hot fields of biology and biomedicine. The MERFISH spatial transcriptomics method has itself been used to elucidate spatial cell type composition in a host of animal tissues (2, 3, 5, 9–16) and has also been commercialized by Vizgen as MERSCOPE. The core methodology has also been used in other spatial transcriptomic methods, for example, in electron microscopy research by (17).

Practically, MERFISH works by encoding all mRNA molecules in a biological sample to be positive for a subset of readout probes, according to a codebook (18). The sample is then incubated with each readout probe in succession, and the combined sequence is read out like a barcode that can be reassigned to corresponding mRNAs using the codebook (Fig. 1A). Since the possible barcodes are all sufficiently different, single errors can be handled without loss of signal.

In a SECDED codebook, each code has at least four differences to each other code, which can be denoted as a minimum Hamming distance (HD) of 4, or minHD4. The practical advantage of this is that if a single error takes place in a code, the resulting nonvalid code can be identified to be one error differing from its originating code, and at least three errors differing from every other valid code, allowing to both detect and correct the single-bit error. If two errors occur, the error can still be detected but not corrected. Most household electronic storage systems use SECDED systems for protection against data corruption (19), and initial MERFISH publications

¹Department of Neuroimmunology, Center for Brain Research, Medical University Vienna, Vienna, Austria. ²Maria Grzegorzewska University, Warsaw, Poland. ³Department of Physiology and Pharmacology, Karolinska Institutet, Stockholm, Sweden.

*Corresponding author. Email: johan.bostroem@meduniwien.ac.at (J.B.); igor.adameyko@ki.se (I.A.)



used a modified version of the same kind of SECDED system to produce filtered extended Hamming codebooks where each studied gene was assigned one code each (3).

When using fluorescent probes against a biological specimen, a 1 bit is the presence of probe and a 0 is the absence of probes, so any background pixel consists of repeated zeros. A binary barcode in a codebook describes which readout probes the corresponding gene should be positive for. The number of positive bits, or the sum of the binary barcode, is called Hamming weight (HW). If an error occurs, it can either be missing a readout probe signal in which case a 1 bit will become a 0 bit after processing, or a noise pixel can be misidentified as a 0 to 1 error. Both cases can be corrected using MERFISH, due to the minimum HD4 requirement.

To avoid differences in signal sensitivity between targeted genes, MERFISH codebooks have a fixed HW. The most commonly used HW is four, as it is high enough to include a high amount of genes but not too high to inflate molecular crowding, where successful microscopic detection can be prevented by too many genes sharing a readout probe and their mRNA colocalizing. Using a fixed HW is also necessary as it does not bias specific mRNA species and it is useful for quality control purposes.

The final parameter that affects codebook size and thus how many mRNA species can be analyzed together is the number of readout probes, or the barcode length, sometimes referred to as number of total bits in the binary barcode. This is the main parameter that affects the complexity of the experimental setup, as most microscopic setups can image between two to four readout probes simultaneously. A high number of readout probes causes longer imaging times and higher risk of failure. We refer to this parameter as barcode length throughout the paper.

Here, codes are processed as sets instead of as binary barcodes. Instead of each code being presented as a string of zeros and ones, we describe the codes as a set of numbers, corresponding to the positions in the binary codebook where they are equal to 1. A binary codebook where each code has v bits can thus be described as a set of subsets of the larger set v , where each 1-bit in the binary string denotes that position as a member. Since we are using fixed HW codebooks, the number of members per subset is equal to the HW. A full codebook can thus be described as a set of subsets of size k , equaling the HW. As an example, the code 0001001000010010 (HW4) can be described as the k -set $\{4,7,12,15\}$ ($k = 4$) which is a subset of the larger set v (1:16), encompassing all the digits between 1 and the length of the binary code. We also make heavy usage of further subsetting k -sets in our codebook-optimizing strategy.

Error-correcting codebooks compatible with MERFISH can be created by using the original algorithm developed by Hamming to protect computer bits using parity bits. Such a book is created for all HWs simultaneously but can be filtered into a set of codes with a constant HW and used for MERFISH, as exemplified in (2), one of the earliest MERFISH publications. This method is however designed to protect existing bits by adding parity bits and is only fully optimized when the number of readout probes is equal to a power of two. In (2), the usage of 16 probes produces a gene list of 140 genes (130 genes plus 10 left open for quality control), which is the highest number of genes achievable for 16 readout probes with a HW of 4. For barcode lengths that are not equal to 2^n , other mathematical methods can be used; however, many sets of parameters do not have an optimal solution.

To evaluate the size of any given codebook, we want to compare the size of the codebook to a mathematically proven theoretical maximum size. These are also called upper bounds, and we make use of the Johnson upper bounds that are designed for constant weight error-correction codebooks (20). These bounds depend on the minHD, HW, and barcode length. As these are upper bounds, they can identify if a given codebook has reached its theoretical maximum. We also make use of it to rank codebooks on a percentage scale, relative to 100% of the Johnson bound for the given parameter combination.

Optimal solutions for constant weight codebooks have traditionally been called Steiner systems (21). A Steiner system is a set of k -sets chosen from 1 to v with no two k -sets sharing the same t -set, where t -sets have one member fewer than k . The last requirement fulfils our minHD requirement. For a HW of 4 and a minimum HD of 4, these combinations are called Steiner quadruple systems, and have been proven to exist for barcode lengths where modulo 6 of v is equal to 2 or 4 (21, 22). They were envisioned in the middle of the nineteenth century (23, 24), and many were found with the accessible use of computers in the 1970s (25). Today, at least one Steiner quadruple system has been identified for all cases relevant for our scope. While there are also equivalent Steiner quintuple systems for HW5 (where modulo 6 of v is equal to 3 or 5, but modulo 5 is not equal to 4), only a few of these have been identified (21). Three Steiner systems have also been identified in our v range for HW6, at v equal to 12, 24, and 36. For any Steiner system, if the barcode length is one less than that of a Steiner system [i.e., congruent to 1 or 3 (mod 6) for HW4], simply removing all quadruplets containing an arbitrarily chosen bit also realizes the Johnson bound (26).

Another field of mathematics that is useful when designing constant weight error-correcting codebooks is the field of (v,k,t) -covering design. It is a field of mathematics that shares an aim similar to Hamming codes but instead of aiming to produce the largest possible set of k -sets without a repeated t -set, it aims at finding the smallest set of k -sets, which comprises all possible t -sets, from the numbers 1 to v (27, 28). This set is called a “lower bound.” While (v,k,t) -covering designs do not abide by any minHD criteria, the lowest identified bound of a (v,k,t) -covering is adjacent enough to the optimal solution of a Hamming codebook to be a useful starting point in a strategy aimed at optimizing Hamming codes. If a Steiner system has been found, it is both a perfect solution to (v,k,t) -covering designs and to constant weight error-robust codebooks.

As most (v,k,t) -covering lower bounds are not directly Hamming compatible due to repeated usage of t -sets, we developed a pruning pipeline that deletes conflicting k -sets, while retaining the highest possible amount of nonconflicting sets. This allows our method to use any (v,k,t) -covering codebook as a starting point for further improvement. Another advantage of (v,k,t) -covering lower bounds is that there exists an exhaustive online database, the La Jolla Covering Repository (29, 30), with lower bounds available as lists of subsets. After pruning, the codebook is analyzed for possibilities for improvement, by investigating networks of unused t -sets and k -sets.

Other results concerning constant weight error codes are cataloged by Brouwer (31) in terms of upper and lower bounds. The upper bound is provided by Johnson; the lower bound is the largest codebook size that has been explicitly constructed or at least non-constructively proven to exist.

No general way of explicitly constructing an optimally large codebook is currently known, but various special-case techniques

have been discovered, taking inspiration from several fields of mathematics, including Galois fields (32), affine geometry (33), group theory (34), game theory (35), and graph theory (36). There are also strategies of conveniently creating powerful (if not necessarily optimal) lower bounds, such as by “shortening” or partitioning some preexisting larger codebook, or by constraining oneself to codebooks that are closed under a given permutation (37).

Much of Brouwer’s database is based on his work alongside Shearer, Sloane, and Smith (37). While their paper has cited a number of construction methods, some codebooks of size 1500 and above could not be given an elegant description and were subsequently “lost.” Today, there are 10 cases in which the largest known codebooks are smaller than the lower bound provided in (37). Three of the previously lost codebooks were reconstructed by Braun *et al.*, who also refined several other lower bounds (38). In our eyes, the existence of lost codebooks showcases the practical utility of an algorithmic approach, in which convenient generation of near-optimal solutions is given precedence over notational elegance.

In published MERFISH literature, many of the above described methods have been used for codebook design. One advantage of MERFISH over SECDED codes for data protection is that while data protection error correcting codes need to protect existing data bits using added parity bits, a MERFISH codebook places no importance on the difference between the two. Therefore, for many parameter combinations of barcode lengths, HW, and minHD, codebooks processed from (v,k,t) -covering theory lower bounds, Steiner systems, or other construction methods of constant weight codes can give substantially higher codebook sizes without any impact on the experimental setup. In the literature of published MERFISH, there are cases of alternative codebook design methods, but many experiments use filtered Hamming code construction (3, 5, 9–11, 13–16, 39). For example, in later MERFISH publications (10, 15), 22 bits are used with 242 genes plus 10 blanks. In (11) 69 bits were used, for the to date largest performed MERFISH experiment, using a 70-bit Steiner system, from the La Jolla covering repository (29, 30) and removing all codes positive for the last bit, leading to 12,903 barcodes.

A challenge when designing an experimental MERFISH setup, especially the first time using a specific combination of HW and barcode length, is the availability of binary codebooks. While websites like Brouwer’s collection presents citations to articles describing lower bounds, many need a deeper mathematical understanding to construct.

Here, we therefore aim to create ready-to-use MERFISH-compatible codebooks for the range of parameters relevant for transcriptomic studies, i.e., from tens to tens of thousands of simultaneous targeted genes. For this aim, we developed an R pipeline that evaluates alternative starting codebooks from the fields described above, processes them to fulfil minHD requirements, and then uses a mathematical approach that analyzes used and unused codes (k -sets) and partial codes (t -sets), and iteratively exchanges sets between the codebook and the unused pool to improve the codebook size.

We hope that making these codebooks available for use will extend the current capabilities of experimental MERFISH setups and allow for a larger set of genes to be explored with no change to hardware or experimental procedure. The method is available as an R script, and we provide precalculated solutions for most of the usable possibilities in the range from tens to low tens of thousands, which is practically relevant for the range of genes in mammalian genomes.

RESULTS

A generalized approach to generate error-robust codebooks

When establishing new MERFISH infrastructure, the number of studied genes will influence the number of decoding probes needed and thus the barcode length of the used codebook. Currently, while a resource for creating encoding probes have been published (3), we identified a need for the systematic generation of optimized codebooks for assigning genes to in the preceding step. We thus created a pipeline to systematically create codebooks for a specified HW and barcode length. The full method is made available as an R function in data S1 and code usage instructions and code documentation is available as text S1. We also supply ready-to-use codebooks for all HW4, HW5, and HW6 codebooks in the range up until a codebook size of ~35,000 targeted genes, covering the range of current biological applications.

The pipeline first calculates the theoretical maximum of an achievable solution, then tests different (v,k,t) -covering lower bounds and starts with the codebook closest to the theoretical maximum. If there is still room to improve, an iterative set exchange loop is initiated, which will continue until running out of room for improvement. The pipeline outputs codebooks in multiple formats ready to use with existing encoding probe generation pipelines.

To compare efficiencies of different methods of codebook generation, it is useful to be able to compare the size of the achieved code set in relation to a mathematical upper limit describing the largest code set possible for given parameters. We make use of the Johnson bound equations (20) and compare each codebook to this upper bound as a way to calculate its efficiency, expressed in percentage units with a maximum of 100% as the Johnson bounds are proven impossible to surpass (20).

Calculated theoretical maximums based on the Johnson Bound are presented in Fig. 1B, for HW4, HW5, and HW6. As these are upper bounds of the theoretical maximum, reaching those bounds is not always possible, but it enables us to evaluate each generated codebook, determine if a codebook has reached its highest possible size, and to compare starting codebooks.

The first step for each parameter combination was to decide on a starting codebook. While the pipeline can generate large codebooks completely de novo, because of the complex space of possibilities of these codebook designs, finding a logical starting codebook seemed critical. As well as reaching a higher codebook size, the running time can also be sped up.

The first starting codebook we decided on was the Steiner systems. For the subset of cases where a Steiner system has been found, it also provides a 100% efficient codebook. Notable, all 100% efficient HW4 filtered extended Hamming codebooks at a barcode length of 2, 4, 8, 16, 32, 64, ..., are also Steiner systems.

Of additional note, for all codebooks generated here, when starting from a Steiner system at a barcode length of one higher, and removing all codes using the highest bit, the generated codebook also has a size at the theoretical maximum, achieving the Johnson upper bound (26). Therefore, for this subset of cases, we used a Steiner system with a barcode length one higher than wanted, and removed all codes using the highest set member. If viewed as binary barcodes, this corresponds to removing each binary code with a 1 in the last digit, then deleting the last digit for the remainder of codes.

For cases where a Steiner system is not available, (v,k,t) -covering lower bounds provide a logical starting codebook. Lowest identified (v,k,t) -covering lower bounds are themselves generated by different

contributors using a host of mathematical methods, but are provided at the La Jolla Covering Repository (30), also published as a dataset (29). They are in a format that can automatically be retrieved by R and processed further. Since a higher-level Steiner system can generate a larger codebook also for smaller barcode lengths, we let the pipeline evaluate multiple higher-level (v,k,t) -covering lower bounds, and choose the starting codebook with the highest starting size after removing codes that conflicted with the minHD criteria. We let the pipeline evaluate every available starting codebook up until the nearest higher available Steiner system, except for the most complex HW5 and HW6 codebooks (with and expected codebook size >20,000), where the three next highest barcode lengths were evaluated instead.

Most (v,k,t) -coverings do not initially fulfil the minHD criteria, so we implemented a pruning algorithm that identifies networks of conflicting codes and deletes them in a way that retains the maximum possible amount of nonconflicting codes.

After identifying a starting codebook and pruning it to achieve the minHD requirements, the pipeline compares the codebook size to the Johnson bound. If it is 100% of the possible size, it is exported as .csv files, both set based and binary based.

If the codebook size is not equal to 100% of the Johnson bound, we developed an iterating pipeline that exchanges subsets of the codes in an existing codebook, aiming at increasing the total codebook size. The iterative pipeline treats each code in the codebook as a subset of a larger set of all possible codes. The first principle that enables this is that a codebook of binary barcodes with a fixed HW can be converted to a set of subsets denoted k -sets (where $k = \text{HW}$) of the members $1-v$, where v is the number of positions in the binary barcode (Fig. 1C).

Treating each code as a set, we can now rephrase the definition of the minHD criteria of extended Hamming codes (with a minimal shared HD of 4): if two k -sets share a subset of size $k-1$ elements (denoted a t -set), they cannot be separated by more than two HD, and are thus not compatible in a minHD4 codebook (Fig. 1D). As an example, the codes/ k -sets $\{1,5,6,7\}$ and $\{3,5,6,7\}$ share the smaller t -set $\{5,6,7\}$, and when converted to binary barcodes, they are only 2HD separated, as visualized in Fig. 1E, which could have been identified by them both containing the t -set $\{5,6,7\}$. Each valid k -set in a minHD4 codebook contains exactly k different t -sets, and none of these t -sets can be used by any other code without breaking the minHD4 criteria (Fig. 1F).

The strategy of identifying potential k -sets by analyzing networks of t -sets is graphically visualized in Fig. 2. Starting from either an empty list or a starting codebook that we wish to increase the size of, we focus on the smaller t -sets, creating a group of all t -sets that are used in the codebook and another group of all unused t -sets. At this stage, all used t -sets are used exactly once or they would break the min4HD criteria. We then calculate their internal networking (with two t -sets sharing an edge if they can both be in the same k -set; for min4HD, this is positive if they share $t-1$ members) and look for groups of connected t -sets. If exactly k t -sets of the unused pool are connected, their corresponding k -set can immediately be added to the codebook as it is not sharing a t -set with any other codebook member, and the codebook has increased in size. Failing that, we focus on all groups of $k-1$ connected t -sets, which we call candidates. Each such group is missing exactly one t -set to form a k -set, and the missing t -set is currently used in the codebook as part of a used k -set. At this point, we can switch out the unused k -set with the

used one, keeping the codebook size static but releasing $k-1$ t -sets into the unused pool, potentially forming a unique k -set with another unused t -set. However, before we use this random approach, we screen through every possible pair of candidates to try to identify a pair that are both missing a t -set used in the same codebook k -set. If that is found, removing that k -set from the codebook will enable us to add both unused k -sets and increase the total size by one. If no pair is found a random set is shuffled, as described previously.

This process iterates until it either has depleted the pool of $k-1$ size t -sets or until it has passed a certain number of iterations without a change to the codebook size. All results here ran for at least 300 iterations after the last size-increasing operation, with the total amount of iterations necessary reaching between 1000 and 3000.

The script is written to perform on any HW, but currently only on min4HD codebooks, which can identify double errors and correct a single error (commonly called extended Hamming codes or SECDED codes). For single-error detection codes, the optimal solutions are always trivial as every nonidentical code with a fixed HW fulfils the min2HD criteria. For any kind of codebook with a fixed HW, every code will always differ with an even HD, so HD3, HD5, etc. are not relevant. A min6HD criteria would allow for double error correction/triple error detection, and while the supplied code and strategy can be extrapolated and repurposed for min6HD codebooks, this paper focuses on the more commonly used extended Hamming codes.

The script runs as an R function, with detailed code instructions provided in the Supplementary Materials. The file also contains full documentation of each function in the associated code, as well as pseudocode for potential repurposing or reuse. The script outputs ready-to-use codebooks as well as a text file containing relevant metadata. To allow for integration with established probe-generating scripts, codebooks are converted to binary, directly compatible with the method to produce encoding probe sets described in (2). A metadata file is also generated, summarizing the number of genes per codebook at each step of the pipeline and measuring the time spent on the distinct steps, as well as saving parameters of the run itself.

Each codebook is also provided in a reordered version that maximizes the HD between the earliest codes. The codes in a minHD4 codebook are all separated by a minHD of 4, but the largest HD between two codes is equal to twice the HW. Ordering the codes in a way that maximizes the HD in the beginning of the codebook allows for gene assignment in descending gene expression order, to make sure that the highest expressed genes are assigned the codes with the largest separated HD. The reordering code first collects the maximum amount of codes not sharing a single barcode position in common, and then scans the remaining codes for the one with the largest overall distance to each code already included, repeating until all codes are included. The entire code is also documented in the Supplementary Materials.

Analyzing the results of produced codebooks

HW4 codebooks have a high density of available Steiner systems, and together with the fact that they can be used to create 100% efficient codebooks for one barcode length smaller, a majority of codebooks generated for HW4 reach 100% efficiency. HW4 Steiner systems are also called Steiner quadruple systems, and they exist for all even barcode lengths that are not divisible by 6. For barcode lengths in the gaps between Steiner systems, codebooks are at least 98% efficient at length 17 and above, and reaches 99.5% efficiency at

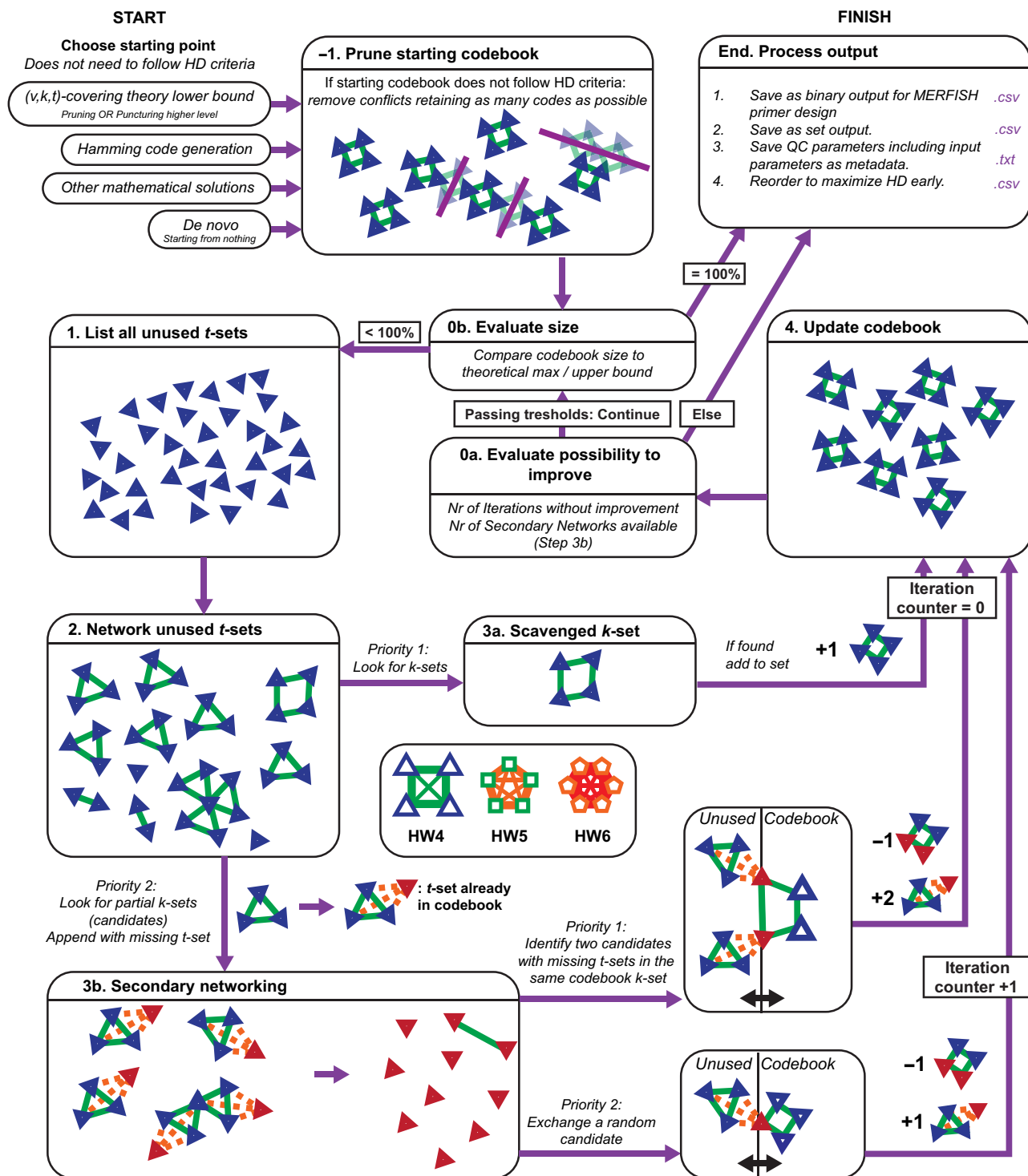


Fig. 2. Iterative pipeline for codebook generation. Summary of the method to generate highly optimized error-robust codebooks. If starting with a non-Hamming-compatible codebook, it is pruned for conflicts (step -1). At this point, the codebook sized is compared to the Johnson bound (step 0b) and if at 100%, the pipeline is finished and results are exported. If below 100%, the iterative set exchange method is initiated. All t -sets not in use in the codebook are summarized (step 1), and their adjacency is mapped out (step 2) to identify possible additions to the codebook (step 3a). If none are immediately found, almost-complete sets of t -sets are investigated for missing a final piece used in the same codebook code (step 3b). If one is identified, it is replaced by both unused nonconflicting codes. If none is found, a random almost-complete set is exchanged with the used code containing its missing t -set. The codebook is then updated (step 4) and if any stop condition has been reached (step 0a-b) the results are outputted as csv files together with metadata. If no stop conditions have been reached, the next iteration begins.

a barcode length of 35 (Fig. 3A and fig. S1A). Available Steiner systems are notated.

When generating codebooks for HW5 and HW6, there is a lower prevalence of Steiner systems. Currently the only Steiner systems for our range of codebook sizes that have been discovered exists at barcode lengths of 11, 23, 35, and 47 for HW5 and at 12, 24, and 36 for HW6. These parameters and one barcode length below thus allowed a straightforward creation of 100% efficient codebooks. For the remainder of combinations for HW5 and HW6, we achieve consistently high results by evaluating all starting codebooks until the nearest Steiner system, then using the iterative set exchange loop to scan for improvements. Produced codebooks for HW5 range from 87 to 95% in efficiency, averaging at 92.9%. HW6 codebooks are of similar quality, with a slightly lower average efficiency at 84.9%, not including 100% results (Fig. 3, B and C, and fig. S1, B and C). As can also be seen from Fig. 3 (B and C), increasing the HW of a codebook can have much larger returns on codebook sizes than increasing the barcode length. To reach a codebook size of 1000 genes, HW4, HW5, and HW6 requires 30, 21, and 18 readout probes, respectively. To reach a codebook size of 5000, the corresponding number of readout probes needed is 51, 30, and 23.

Compared the results to filtering of Hamming codebooks for HW4 codebooks, we see the greatest yield immediately succeeding a power of two, as shown in Fig. 3 (D and E). Our method shows a 90% increase in the number of genes that can be included immediately after a barcode length of a power of two, then slowly decreasing in increase in efficiency until the next power of two.

We also wanted to compare our results to the highest proven lower bounds collected by Brouwer (31), as they are a comprehensive resource summarizing highest available proofs for lower bounds of constant weight codes. Results can be seen in Fig. 3 (F to H). While a majority of produced codebooks are close to these lower bounds, two specific codebooks manage to surpass them. For HW5, producing codebooks with barcode lengths of 26 and 29, respectively, our produced codebooks surpassed the previously reported lower bounds by 6 codes and 1 code, respectively. Although this accounted for a minor practical increase of 0.2 and 0.02%, respectively, we wanted to investigate if this was the limit of our method for these two examples. We therefore reran both codebook pipelines with an increased number of allowed iterations after the last detected change (up to 5000), but we did not see any further improvement. We also tried running both pipelines 10 times using 1000 maximum iterations, which recreated the same codebook size 9 of 10 times, but did not succeed in increasing these two codebooks further.

All generated codebooks can be found in data S3, S4, and S5, for HW4, HW5, and HW6, respectively. The results are available both in binary format and set-based format. Codebooks are also provided in a reordered version that maximizes the HD at the start of the codebook, with each subsequent code optimized to minimize the HD to all codes above. This enables assigning gene targets in order of prior known gene expression values, to minimize the number of shared encoding probes between targeted genes with high expression.

Quality control evaluations

To showcase the variability of the iterative section of the codebook generation, we generated 50 codebooks for barcode length 21, HW4 and 23 codebooks for barcode length 37, HW4. For both tests, we used a (v, k, t) -covering with equivalent parameters. The distribution of repeated generation of these codebook sizes can be seen in fig. S1 (D to F).

The pipeline does not make heavy use of CPU power as it runs on a single processor. All generated codebooks were created on an Intel Xeon CPU E5-2699 version 4 processor, with 2.2G Hz processing speed. Using a faster processor can linearly decrease running time. There is a possibility to use parallel processing during the pairwise candidate searching to speed up more complex cases, but as almost all of our codebooks could be generated over a single overnight run, we did not see the practical need to implement it.

RAM usage is mainly dependent on the size of initial and produced codebooks, especially when creating the HD relationship matrix between all codes. However, we managed to create up to 40 codebooks (among the provided HW5 and HW6) codebooks simultaneously on the above mentioned processor, staying below a combined RAM usage of 200 GB. The RAM usage is heaviest at the very start of a pipeline, during the initialization and pruning, so staggered starts of simultaneous runs can alleviate RAM usage concerns.

The main resource used is processor time. Time spent to produce a codebook depends heavily on what starting codebook is available. To give an indication of the relationship between complexity of barcode and time spent, we provide a graph showing the time spent for all codebooks generated in this resource. In fig. S2A, time is plotted against barcode length, and in fig. S2B, time is plotted against codebook size.

Because of the nature of the iterative design, no codebook can exit the pipeline without being compliant with the minHD criteria. However, we still want to provide an R function to verify the minimum HD of any provided csv file, including all codebooks generated here. It is provided as a separate function in data S1 and identifies if a supplied .csv file is based on binary or set notation, then calculates the minimum HD, and if not overly complex, generates a heatmap of the HD relationships within the codebook. A few examples are provided in fig. S2C.

Increasing the HW and investigating its effect on molecular crowding

Increasing the HW comes with a large advantage in the reduction of necessary readout probes (thus lowering imaging time) but it does however come with a practical limitation in the form of increased density of probes (molecular crowding) and subsequent loss of reads due to sources of light being too close together for separation. While the total read count of a MERFISH experiment is always positive to increase, the molecular crowding (i.e., how many mRNA reads share a given acquisition area for a single decoding probe) can be detrimental if it is large enough where multiple mRNA appears in the same pixel. We wanted to estimate this effect, using theoretical maximum values as an input.

Molecular crowding scales linearly with the number of genes divided by the barcode length, as the number of decoding probes is equal to the length of the barcodes. As HW5 and HW6 scales up their codebook sizes quicker than HW4, it also leads to a sudden increase in molecular crowding. Figure 4A showcases the relative increase in molecular crowding with increased barcode length, and Fig. 4B shows the molecular crowding relative to the number of studied genes. Both graphs are normalized to the probe density of 16Bit HW4 MERFISH and use the Johnson bound as input. For many practical applications, the number of pixels per area cannot be increased, and thus increasing the HW can lead to an increased molecular crowding effect and the loss of read integrity, which can be avoided by staying with HW4 and increasing the barcode length.

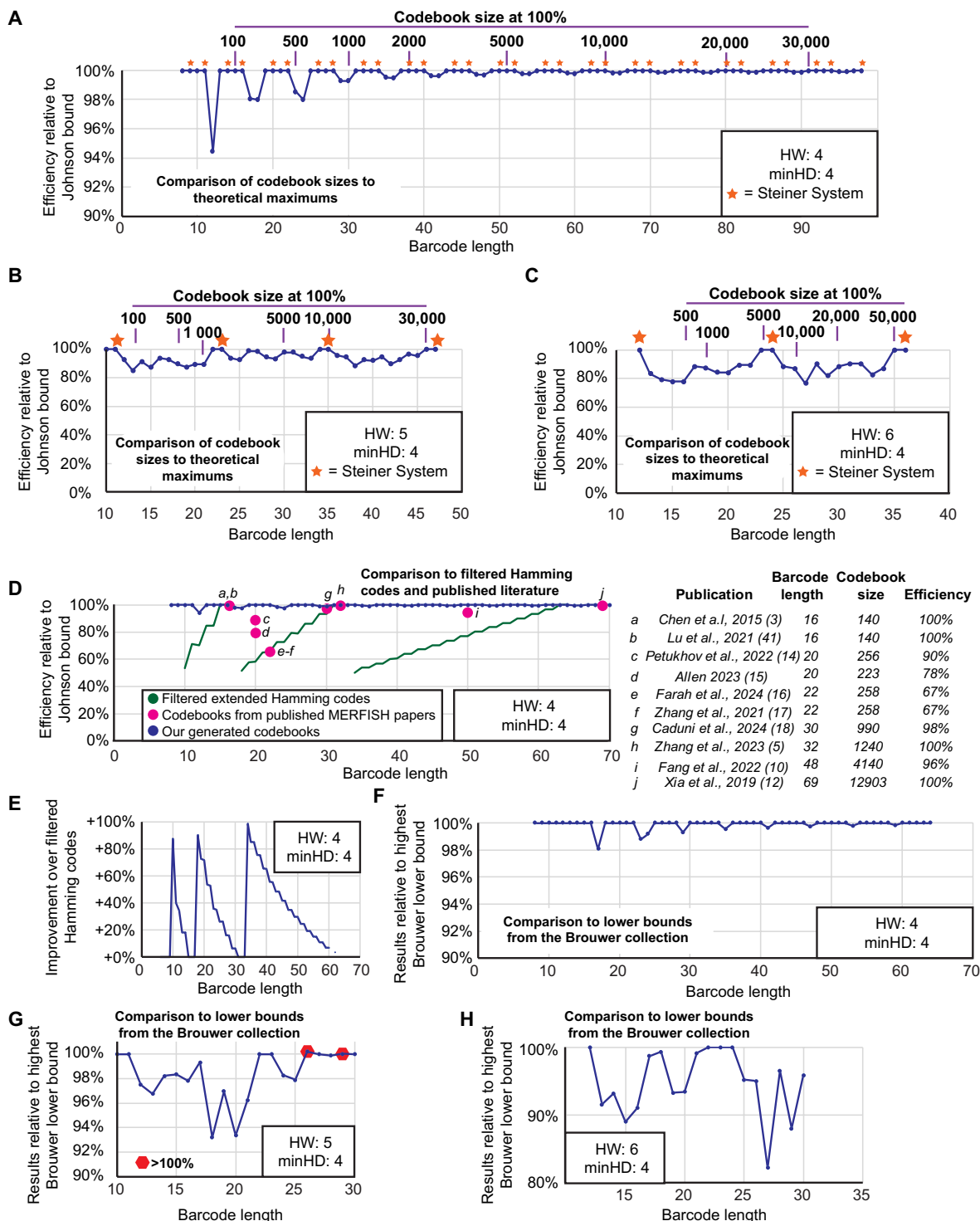


Fig. 3. Results of codebook generation. (A to C) Results from generating codebooks for HW 4 (A), HW5 (B), and HW6 (C), spanning codebooks to investigate gene set sizes from low double digits to tens of thousands. Available Steiner systems are highlighted. Results are presented in relation to the theoretical maximum at the same barcode length, using the Johnson bounds. (D and E) Results for HW4 are compared to filtered Hamming Codes and examples from MERFISH literature, annotated using shortened references. (F and G) A comparison of the results to previously highest reported lower bounds for HW4 (F), HW5 (G), and HW6 (H), as collected by A. E. Brouwer. Two codebooks surpassing these lower bounds are highlighted.

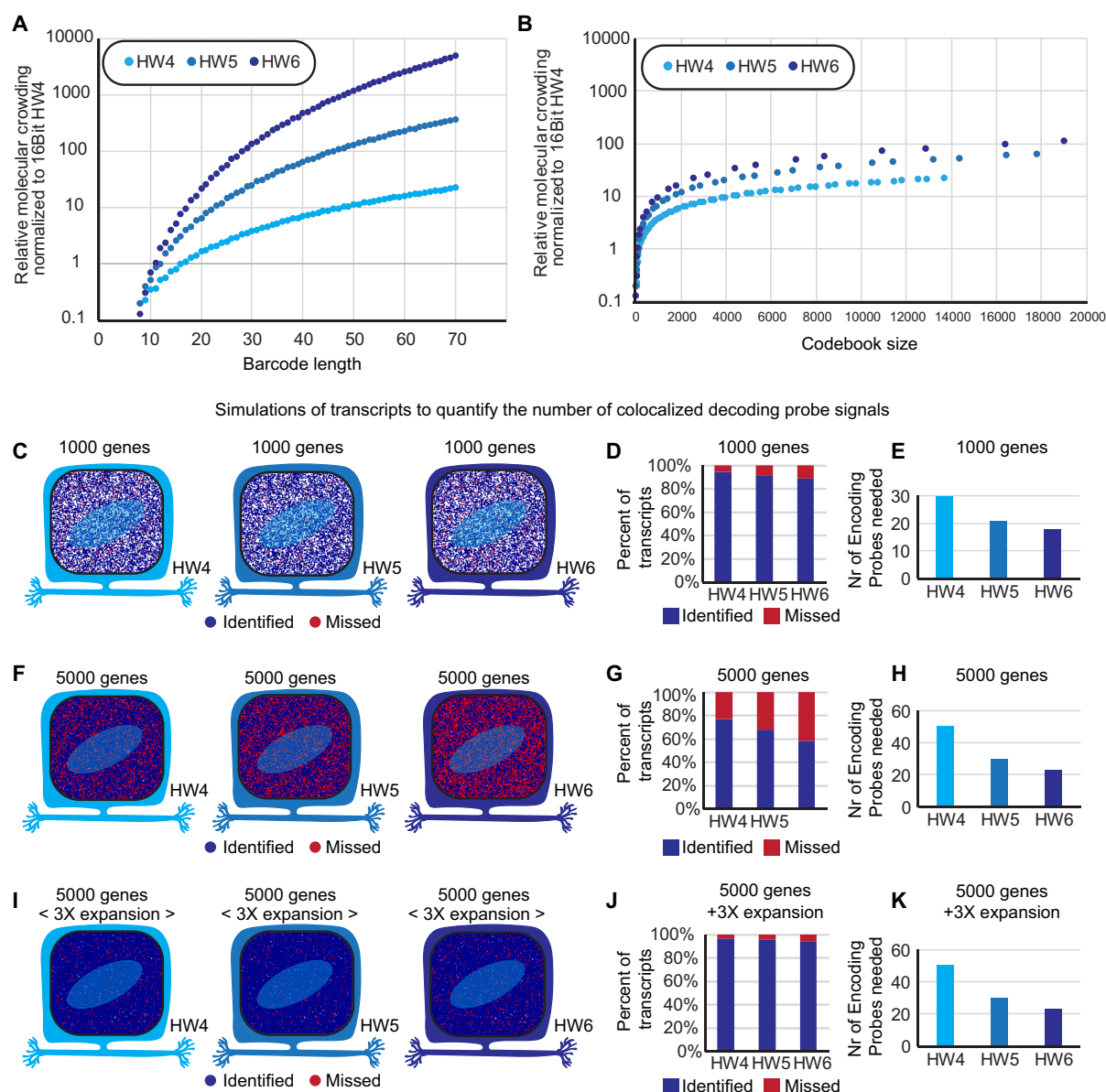


Fig. 4. Relationship between Barcode lengths, HWs, and molecular crowding. (A) Relative probe density for a single image expressed as relative number of probes per pixel, relative to a 16Bit HW4 experiment. The relative molecular density is plotted against increased barcode length, for HWs of 4, 5, and 6. (B) Relative probe density for a single image expressed as number of probes per pixel, relative to a 16Bit HW4 experiment, but plotted against the theoretically maximum gene set size. (C to K) Simulated quantification of missed decoded transcripts due to shared decoding probes being present closer than the diffraction limit for various practical scenarios over different HW codebooks. Transcripts are simulated in a single cell, quantified as a percentage of total reads, and compared to the practical advantage of needing fewer decoding probes for the scenario. Results are presented for a codebook of 1000 genes [(C) to (E)], a codebook of 5000 genes [(F) to (H)], and a codebook of 5000 genes on a sample expanded three times using expansion microscopy [(I) to (K)].

However, using HW5 and HW6 codebooks gives large advantages in lowering the number of readout probes needed for a given gene set, which also lowers microscopy time and can allow for a MERFISH infrastructure to handle more experiments in a given time frame. As long as the effect on molecular crowding is taken into account, higher HW codebooks can be very useful.

To showcase how the effect of increased molecular crowding shown in Fig. 4 (A and B) can affect a practical scenario, we simulated the effect of increasing the density of decoding probes to estimate the number of conflicts and lost transcripts due to the same

probe appearing in two reads closer than the diffraction limit. We simulated the mRNA content of a simulated cell and then simulated the output of different HW codebooks for a specific gene set. For the full number of transcripts per cell, we assume 100,000 of available reads per cell for a full transcriptome, based on the data on U2OS cells in Xia *et al.* (11) where the mean number of reads per cell was 92,000 while targeting a majority of the expressed transcriptome. For gene distributions, we used previously published single-cell data (40) and chose the average gene expression values for a population of sensory neurons as the basis for transcriptome distribution inside

a simulated cell. Two random sets of genes were chosen for analysis out of the genes expressed, one with 1000 genes, and the other with 5000.

The main outcome measurement was the percentage of mRNAs affected by having a shared decoding probe present at a distance closer than the diffraction limit, as this is the type of conflict that can increase with higher HWs. We use the emission wavelength close to green fluorescent protein (~500 nm) together with a numerical aperture of 1.4 to calculate the diffraction limit.

Assigning the chosen set of genes to codebooks with similar sizes from all generated HWs, we can quantify the effect of the increased probe density by checking one decoding probe at a time, and summarizing the number of transcripts affected. The total number of nondetected transcripts for various combinations of targeted gene sizes and HW are visualized in Fig. 4 (C to H).

In an actual MERFISH experiment, the number of detected reads depend on a few more factors, due to both the error robustness of the codebook design, the choice of decoding algorithm, and the stability and consistency of the imaging used. The values provided here focus on the direct effect of having a higher likelihood to share a decoding probe in common when increasing the HW, providing a ballpark estimate of when increasing the HW can have too high effects on the number of detectable reads, and when it is safe to increase the HW to decrease imaging time.

Two reads that overlap can also be problematic when they do not share a decoding probe, as they can be decoded as one read with twice the number of positive probes. This type of conflict will however not change depending on HW. These could however account for some part of the increase in nondetected transcripts when the HW is increased, so the practical increase could be smaller than presented.

With a codebook size of 1000 genes, the number of nondetected transcripts comprise a small fraction (Fig. 4, C to E), but when increasing the codebook size, we can see the effect increasing, combined with increasing HW (Fig. 4, F to H). Increasing the effective resolution threefold (for example by using expansion microscopy), the effect can be alleviated and allow to transition to higher HW to save considerable imaging time (Fig. 4, I to K).

In summary, we generated a resource of ready-to-use codebooks for multiple HWs for multiplexed biological applications including MERFISH. We created a pipeline to systematically generate codebooks for a wide range of codebook sizes for HW4, HW5, and HW6, for codebook sizes up to multiple tens of thousands sufficient to cover even full-transcriptome studies. While HW4 is the most common HW used today in MERFISH projects, HW5 and HW6 codebooks can be used to reduce imaging time of MERFISH projects, as long as the molecular crowding effects is not too large. Transitioning to HW5 or HW6 can also enable a larger amount of genes studied for a given timeframe by decreasing the number of decoding probe rounds needed for a certain codebook size. As novel microscope techniques are generated, pushing the diffraction limit and expansion microscopy methods separating transcripts even before imaging, we believe that there will be a larger role for higher HW utilization in MERFISH methodologies and other multiplexed techniques, and we hope that our codebooks will become a key resource.

DISCUSSION

In the early 2010s, as in situ hybridization methods became more and more multiplexed, error-robust codes were successfully used to

unlock large-scale spatial transcriptomic experiments, establishing the MERFISH methodology (2, 3). Early MERFISH papers used filtered extended Hamming codes, using similar methodology to consumer electronic data protection for the generation of codebooks (3, 4, 19), which unlocked the power of error-robust codebooks to facilitate detection and correct calling of transcripts without full sensitivity of each specific readout probe. In contrast to the practical implementation in the field of data protection, MERFISH provides a relatively unique case of a practical implementation of constant weight error-robust codes that are useful by themselves, without any prior data bits to protect. This places a higher emphasis on finding the largest codebooks possible for a given set of experimental parameters.

The field of constant weight error-robust codebooks is 180 years old (23, 24) and while many mathematical solutions including Steiner systems and proofs of lower bounds exist for many parameter sets, the focus has been on construction methods and elegant designs. Many papers have been published submitting innovative solutions or theoretical lower bounds a few sets of parameters at a time, making a systematic overview and creation of ready-to-use codebooks for a practical purpose challenging. To systematically create codebooks for a wide array of parameter designs, one would need to read, understand, and often reconstruct codebooks using a variety of methods, many iterating on each other (25, 26, 37, 41, 42).

We therefore aim to provide a pipeline that combines choosing a logical starting codebook that does not need to follow the rules of minHD requirements, and a greedy algorithm for improvement, that can generate codebooks within a reasonable time frame. The computational time ranges from minutes for results covering hundreds of genes, to multiple days for tens of thousands, but no special hardware is required as the pipeline can be run on a single processor.

Here, we provide a systematic method to generate error-robust codebooks fulfilling a minHD criteria of 4, enabling double error detection and single error correction of readout data. We provide ready-to-use codebooks but also the pipeline itself to generate more codebooks, and the output is directly compatible with established methods. The code is limited to producing minHD4 codebooks, but can be repurposed for higher error-detection levels.

The pipeline begins by parsing a logical mathematical starting codebook, then prunes it to ensure it abides by the minHD criteria. After pruning, the pipeline evaluates if there is any possibility to increase it, and if so, performs an iterative search characterizing networks of partial codes and exchanges codes in a manner to increase the final codebook size. When we systematically produced codebooks for a range of genes relevant for spatial transcriptomics, we made use of a resource of (v, k, t) -covering lower bounds (29) as our source of starting codebooks, which consistently provided high results of final codebook sizes.

The iterative strategy uses randomization while exchanging sets, leading to a small difference when generating multiple codebooks with the same parameters. This is due to the fact that random decisions early in the iterative code can change the possibilities further on. The end results usually vary in the low single digits but when establishing a spatial transcriptomics method, we recommend running the pipeline multiple times and using the highest achieved result.

Our results show a large increase in the number of genes that can be investigated with established infrastructure, when compared to filtered extended Hamming codes. The increase is largest after a

readout barcode length of any power of two, due to common usage of filtered extended Hamming codes, which give lower codebook sizes in these regions. Notably, this encompasses a large increase in the number of genes between 140 genes and 1000 genes, a range which is of great interest for biological experiments and that can enable current experimental setups to increase the number of genes studied substantially. The second considerably improved range is that from 1500 genes studied up to multiple thousands, which we envision will also be highly used with future spatial transcriptomic methodologies.

We would not call the resulting codebooks “elegant,” compared to case-by-case mathematical constructions that are based on a deep understanding of set theory, but our method provides a straightforward and efficient method to generate codebooks for use in the field of molecular biology and elsewhere, without requiring prior knowledge of the mathematical field for the user.

However, among the codebooks generated throughout this paper, two managed to surpass the highest previously reported lower bound (31). The improvement over the previously highest proof might be marginal from a practical standpoint, but we hope that it can highlight the value of iterative-based approaches in solving complex mathematical problems and maybe provide insights for future theoretical work.

With the rapid growth of other omics methods, and modern biological and medicinal research collecting larger and larger datasets (43), we would not be surprised if other methods could make use of constant weight error robust codebooks. If so, we hope that our method can be useful in making them accessible to a wider field.

Changing the HW of used codebooks can have large advantages, primarily in reducing microscope time. The downside to increasing the HW is the higher density of signal in each microscopic readout but as long as this downside does not overwhelm the decoding analysis pipeline, the advantages can minimize experimental time substantially. We simulated a few specific practical scenarios to showcase the effect when transitioning to higher HWs, as well as when increasing the effective resolution. The simulated values provide a rough estimate as the exact numbers of missed reads depend heavily on the exact microscopic infrastructure and choice of decoding algorithm, but showcase the effect of increased HW together with increased codebook sizes. With larger numbers of genes studied, the molecular crowding will increase regardless, and increasing the optics of the acquiring microscope also has a cost in running time. Higher objectives have a lower field of view necessitating more images to cover the same area, and diffraction-unlimited imaging techniques can also increase the imaging time. Recent advantages in the field of expansion microscopy could also prove useful in separating neighboring signal. When possible, transitioning to using higher HW for MERFISH codebook designs could help to reduce the overall acquisition time and enable a faster throughput on a given experimental setup.

To conclude, we present a method to boost the multiplexing capability of established error-robust spatial transcriptomic methods by systematically generating highly efficient codebooks. We also provide finished codebooks compatible both with current methodologies such as MERFISH but also with other methods that use error robustness. Our results provide an easy-to-use resource that can be used in the design of MERFISH probe sets and future error-robust methodologies.

MATERIALS AND METHODS

Experimental design

Our strategy to generate error-robust codebooks was implemented in R (version 4.3.2) (44). In R, the libraries dplyr (45) and rvest were used. For comparison to filtered full Hamming codes, we expanded on previously published code in MATLAB (cR2021a).

The supplied pipeline is provided as a folder of R functions (data S1), documented in the Supplementary Materials, and can be applied to any codebook size, but we created codebooks up until gene set sizes of tens of thousands, based on the upper range of the number of unique mRNAs targeted in spatial transcriptomic studies.

Calculation of filtered extended Hamming code sets

Filtered extended Hamming codebooks were created for every barcode length up to 64 using a modified version of the code provided in (2). Because of the massive scale of generating a 64-bit codebook (requiring either 1000 petabytes RAM or thousands of years of CPU time), we preconstructed every code before parity bit addition with less or equal than 4HW before running through the Hamming generator, appending parity bits, then filtered the resulting codes once more based on HW.

The preconstruction was performed by three repeated additions of every element of the series 2^n where $n = (0, 1, 2, 3 \dots 63)$, stored in an unsigned 64-bit integer. Each unique addition was then converted into binary, and filtered for HW. The modified MATLAB code is available as data S2.

Calculation of theoretical maximums

Theoretical maximums for minHD4 binary codebooks were calculated using the following Johnson bound equation, implemented in R.

Johnson bound(n , minHD, HW) =

$$\left\lfloor \frac{n}{HW} + c \left\lfloor \frac{n-1}{HW-1} + c \left[\dots \left\lfloor \frac{n-HW + \left\lceil \frac{\text{minHD}}{2} \right\rceil}{\left\lfloor \frac{\text{minHD}}{2} \right\rfloor} + c \left[\dots \right] \right] \right\rfloor \right\rfloor \right\rfloor \quad (1)$$

For implementation in R, c had to be added to the original equation as a correction factor to prevent rounding a value of repeating #0.999... to # instead of # + 1, which happens in a small subset of cases. We used a value of 0.000001 for c throughout.

Starting options

The provided R function can be executed in five different ways. The first way is to run it using de novo, i.e., without a starting codebook. The second way is a user-supplied codebook, and the R code can identify if the provided codebook is in binary format or based on sets. The third way is to use a (v, k, t) -covering lower bound from <https://ljcr.dmgordon.org/cover/table.html> (29, 30) with the parameters of v , k and t , as barcode length, HW, and HW-1. The fourth way is to use a (v, k, t) -covering lower bound with a higher v than barcode length, and the code will automatically discard any codes using members of v higher than the barcode length. The fifth way is to let the pipeline test every (v, k, t) -covering lower bound up until a designated upper limit, puncturing and pruning each of them before comparing them. As a logical upper limit, we recommend the nearest highest available Steiner system, but because of computational demands, stopping one or two steps higher can be the pragmatic option. The metadata provided with each generated codebook (data S3 to S5) also contains information on which v proved to be the best starting codebook for the entire range of

analyzed codebooks, and it most often is the codebook with v one higher than the barcode length.

Set pruning

When using a (v, k, t) -covering lower bound as a starting codebook, most solutions will be breaking the HD rule of Hamming sets due to the numbers of k -sets not being divisible by the number of t -sets and therefore appearing in more than one k -set, so some pruning is required to turn optimal solutions from the covering field into Hamming code sets. There are many ways to trim down the same codebook, but because the solution is hypothetically rather close to an optimized Hamming set, the pipeline identifies each network of codes with a minimum HD lower than the criteria and eliminates codes from the network in a manner retaining the highest amount.

This iterates until the set has the required minimum HD requirement. The decision of which code to prune is made with the intent to retain as many codes as possible, and it is performed fully optimized up to a networking level of three, but for any more complicated networks, a random code with the highest amount of conflicts is chosen for removal in each iteration. The pruning code is integrated in the iterative pipeline and is automatically detected and used if necessary.

We acquire (v, k, t) -covering lower bounds from <https://ljcr.dmgordon.org/cover/table.html> (29, 30). The R script automatically acquires the appropriate lower bound for the parameters requested and identifies if the downloaded codebook requires pruning. It then compares the length of the codebook to the theoretical maximum and if it is lower, it initiates the iterative set exchange method to increase the codebook size. For certain parameter combination, the script can also download every (v, k, t) -covering up until a user provided cutoff (we recommend the next available Steiner system or to reduce computational intensity one or two barcode lengths above the requested barcode length).

Iterative set exchange method

The iterative set exchange method starts by defining all possible t -sets, categorizing them as either used or unused. It then scans through the set of unused t -sets for potential candidates. This is technically performed by creating a hypothetical list of all possible k -sets that each t -set could be a part of, before counting up how many t -sets each such k -set has available in the list of unused t -sets. In the R script, this list is annotated as “ghost k -sets,” as they are potential but not yet in use.

Any ghost k -set with the full complement of t -sets available in the unused t -set list can be directly added to the codebook, and this is the first scan that is performed after the ghost k -set table is created, and is referred to as “EasyPickings.” If an EasyPicking is identified, it is added to the codebook and the script restarts with the recalculation of used and unused t -sets.

Any ghost k -set with exactly one missing t -set is referred to as a “candidate” and all candidates are collected for the next step. The next step aims to identify two candidates that are missing t -sets currently used in the same codebook k -set. This is achieved by creating a secondary set of hypothetical k -sets (“double-ghost k -sets”) that is created exactly as the first hypothetical set but using the missing t -sets as input. This set is then filtered on the basis of how many primary ghost k -sets are linked to it, with any double-ghost k -set with at least two primary ghost k -sets that are moved forward to the final check. One by one, the double-ghost k -set is checked for pairwise primary ghost k -sets, to see if any pair is not conflicting with each other. If such a pair

is found, the double-ghost k -set can now be removed from the codebook and both primary ghost k -sets can be added without creating a conflict. After adding, the pipeline starts from recalculation of the used and unused t -sets. Computationally, a majority of the double-ghost k -sets identified with multiple associated primary ghost k -sets are actually conflicting, so this step is computationally the most time-consuming part of the script.

If no pairwise primary ghosts are conflict-free after checking each possible double-ghost k -set, the script reverts to the final option, which is to choose a random candidate and add it to the codebook, while removing the code that currently used its missing t -set, followed by starting with the recalculation steps. If this step is performed a certain number of iterations in a row without any improvements to codebook size (300 iterations for all of our generated codebooks, but included as an optional user-provided argument in the R function), the pipeline stops and processes output files.

On the basis of the experience creating the codebooks published here, 10 to 100 random set exchange iterations are usually needed after the initial burst of EasyPickings before the first double-ghost k -set is identified. Often, once the first double-ghost k -set is identified, many more follow in rather quick succession. We believe that this is because of the internal structures in the starting codebooks that need some amount of randomization, before allowing more codes to be added.

Statistical analysis

No statistical analysis was used except for descriptive statistics. Each generated codebook was generated once for each starting codebook. Figure S1B shows the distribution in codebook generation results over 50 iterations for codebooks of two different sizes and from a de novo design.

Simulated analysis of conflicts due to shared decoding probes

The transcriptome of a cell was simulated using single-cell sequencing read count distributions of a sensory neuron from Kastri et al. (40). The total read count was approximated to 100,000 reads based on the almost-full transcriptome results from Xia et al. (11) and 100,000 reads following the sensory neuron gene distributions were generated in a 100- μ m-wide square. After simulating the positions of these transcripts, codebooks were generated by randomizing a specific number of genes (1000 and 5000) and assigning them to HW4, HW5, and HW6 codebooks with at least the same amount of codes. Conflicts were then detected by checking colocalization one decoding probe at a time (a conflict was defined as a colocalization closer than the diffraction limit for a fluorophore at 500 nm using an objective with a numerical aperture of 1.4) and the total number of conflicting reads were summed up. The percentage of all reads that experienced such a conflict was quantified as the output variable and compared to time saved by reducing the number of imaging cycles due to the lower amount of decoding probes needed.

Supplementary Materials

The PDF file includes:

R Code Documentation
Figs. S1 and S2
Legends for data S1 to S5

Other Supplementary Material for this manuscript includes the following:

Data S1 to S5

REFERENCES AND NOTES

1. L. Moses, L. Pachter, Museum of spatial transcriptomics. *Nat. Methods* **19**, 534–546 (2022).
2. J. R. Moffitt, J. Hao, G. Wang, K. H. Chen, H. P. Babcock, X. Zhuang, High-throughput single-cell gene-expression profiling with multiplexed error-robust fluorescence in situ hybridization. *Proc. Natl. Acad. Sci. U.S.A.* **113**, 11046–11051 (2016).
3. K. H. Chen, A. N. Boettiger, J. R. Moffitt, S. Wang, X. Zhuang, Spatially resolved, highly multiplexed RNA profiling in single cells. *Science* **348**, doi: 10.1126/science.aaa6090 (2015).
4. R. W. Hamming, Error detecting and error correcting codes. *Bell Syst. Tech. J.* **29**, 147–160 (1950).
5. M. Zhang, X. Pan, W. Jung, A. R. Halpern, S. W. Eichhorn, Z. Lei, L. Cohen, K. A. Smith, B. Tasic, Z. Yao, H. Zeng, X. Zhuang, Molecularly defined and spatially resolved cell atlas of the whole mouse brain. *Nature* **624**, 343–354 (2023).
6. A. Chen, S. Liao, M. Cheng, K. Ma, L. Wu, Y. Lai, X. Qiu, J. Yang, J. Xu, S. Hao, X. Wang, H. Lu, X. Chen, X. Liu, X. Huang, Z. Li, Y. Hong, Y. Jiang, J. Peng, S. Liu, M. Shen, C. Liu, Q. Li, Y. Yuan, X. Wei, H. Zheng, W. Feng, Z. Wang, Y. Liu, Z. Wang, Y. Yang, H. Xiang, L. Han, B. Qin, P. Guo, G. Lai, P. Muñoz-Cánoves, P. H. Maxwell, J. P. Thiery, Q. F. Wu, F. Zhao, B. Chen, M. Li, X. Dai, S. Wang, H. Kuang, J. Hui, L. Wang, J. F. Fei, O. Wang, X. Wei, H. Lu, B. Wang, S. Liu, Y. Gu, M. Ni, W. Zhang, F. Mu, Y. Yin, H. Yang, M. Lisby, R. J. Cornall, J. Mulder, M. Uhlén, M. A. Esteban, Y. Li, L. Liu, X. Xu, J. Wang, Spatiotemporal transcriptomic atlas of mouse organogenesis using DNA nanoball-patterned arrays. *Cell* **185**, 1777–1792.e21 (2022).
7. Y. Deng, M. Bartosovic, P. Kukanja, D. Zhang, Y. Liu, G. Su, A. Enninfel, Z. Bai, G. Castelo-Branco, R. Fan, Spatial-CUT&Tag: Spatially resolved chromatin modification profiling at the cellular level. *Science* **375**, 681–686 (2022).
8. R. Arora, C. Cao, M. Kumar, S. Sinha, A. Chanda, R. McNeil, D. Samuel, R. K. Arora, T. W. Matthews, S. Chandarana, R. Hart, J. C. Dort, J. Biernaskie, P. Neri, M. D. Hycrca, P. Bose, Spatial transcriptomics reveals distinct and conserved tumor core and edge architectures that predict survival and targeted therapy response. *Nat. Commun.* **14**, 5029 (2023).
9. R. Fang, C. Xia, J. L. Close, M. Zhang, J. He, Z. Huang, A. R. Halpern, B. Long, J. A. Miller, E. S. Lein, X. Zhuang, Conservation and divergence of cortical cell organization in human and mouse revealed by MERFISH. *Science* **377**, 56–62 (2022).
10. M. Zhang, S. W. Eichhorn, B. Zingg, Z. Yao, K. Cotter, H. Zeng, H. Dong, X. Zhuang, Spatially resolved cell atlas of the mouse primary motor cortex by MERFISH. *Nature* **598**, 137–143 (2021).
11. C. Xia, J. Fan, G. Emanuel, J. Hao, X. Zhuang, Spatial transcriptome profiling by MERFISH reveals subcellular RNA compartmentalization and cell cycle-dependent gene expression. *Proc. Natl. Acad. Sci. U.S.A.* **116**, 19490–19499 (2019).
12. J. R. Moffitt, D. Bambach-Mukku, S. W. Eichhorn, E. Vaughn, K. Shekhar, J. D. Perez, N. D. Rubinstein, J. Hao, A. Regev, C. Dulac, X. Zhuang, Molecular, spatial, and functional single-cell profiling of the hypothalamic preoptic region. *Science* **362**, eaau5324 (2018).
13. V. Petukhov, R. J. Xu, R. A. Soldatov, P. Cadinu, K. Khodosevich, J. R. Moffitt, P. V. Kharchenko, Cell segmentation in imaging-based spatial transcriptomics. *Nat. Biotechnol.* **40**, 345–354 (2022).
14. W. E. Allen, T. R. Blosser, Z. A. Sullivan, C. Dulac, X. Zhuang, Molecular and spatial signatures of mouse brain aging at single-cell resolution. *Cell* **186**, 194–208.e18 (2023).
15. E. N. Farah, R. K. Hu, C. Kern, Q. Zhang, T. Y. Lu, Q. Ma, S. Tran, B. Zhang, D. Carlin, A. Monell, A. P. Blair, Z. Wang, J. Eschbach, B. Li, E. Destici, B. Ren, S. M. Evans, S. Chen, Q. Zhu, N. C. Chi, Spatially organized cellular communities form the developing human heart. *Nature* **627**, 854–864 (2024).
16. P. Cadinu, K. N. Sivanathan, A. Misra, R. J. Xu, D. Mangani, E. Yang, J. M. Rone, K. Tooley, Y. C. Kye, L. Bod, L. Geistlinger, T. Lee, R. T. Mertens, N. Ono, G. Wang, L. Sanmarco, F. J. Quintana, A. C. Anderson, V. K. Kuchroo, J. R. Moffitt, R. Nowarski, Charting the cellular biogeography in colitis reveals fibroblast trajectories and coordinated spatial remodeling. *Cell* **187**, 2010–2028.e30 (2024).
17. P. Androvic, M. Schifferer, K. Perez Anderson, L. Cantuti-Castelvetri, H. Jiang, H. Ji, L. Liu, G. Gouna, S. A. Berghoff, S. Besson-Girard, J. Knoferle, M. Simons, O. Gokce, Spatial transcriptomics-correlated electron microscopy maps transcriptional and ultrastructural responses to brain injury. *Nat. Commun.* **14**, 4115 (2023).
18. J. R. Moffitt, X. Zhuang, *RNA Imaging with Multiplexed Error-Robust Fluorescence in Situ Hybridization (MERFISH)* (Elsevier Inc., ed. 1, 2016; <http://dx.doi.org/10.1016/b978-0-12-407030-0.ch020>) vol. 572.
19. P. K. Kythe, Dave K.; Kythe, “Extended Hamming Codes” in *Algebraic and Stochastic Coding Theory* (CRC Press, 2017), pp. 95–116.
20. S. M. Johnson, A new upper bound for error-correcting codes. *IEEE Trans. Inf. Theory* **8**, 203–207 (1962).
21. C. J. Colbourn, R. Matheron, “Steiner Systems” in *Handbook of Combinatorial Designs* (2007), pp. 102–110.
22. H. Hanani, On quadruple systems. *Can. J. Math.* **12**, 145–157 (1960).
23. W. S. Woolhouse, “Prize Question No 1733” in *Lady’s and Gentleman’s Diary* (1844).
24. J. Steiner, Combinatorische Aufgabe. *J. Reine Angew. Math.* **45**, 181–182 (1853).
25. C. C. Lindner, R. Alexander, Steiner quadruple systems – A survey. *Discret. Math.* **22**, 147–181 (1978).
26. L. Ji, Asymptotic determination of the last packing number of quadruples. *Des. Codes Cryptogr.* **38**, 83–95 (2006).
27. D. M. Gordon, D. R. Stinson, “Coverings” in *Handbook of Combinatorial Designs* (CRC Press, 2006), pp. 391–398.
28. D. M. Gordon, O. Patashnik, G. Kuperberg, New constructions for covering designs. *J. Comb. Des.* **3**, 269–284 (1995).
29. D. M. Gordon, Data Set: La Jolla Coverings Repository (v1.0). Zenodo (2024); <https://doi.org/https://doi.org/10.5281/zenodo.10779737>.
30. D. M. Gordon, Covering Designs; <https://dmgordon.org/cover/>.
31. A. E. Brouwer, Bounds for Binary Constant Weight Codes; <https://aeb.win.tue.nl/codes/Andw.html>.
32. F. W. Fu, A. J. Han Vinck, S. Y. Shen, On the constructions of constant-weight codes. *IEEE Trans. Inf. Theory* **44**, 328–333 (1998).
33. V. D. Tonchev, Maximum disjoint bases and constant-weight codes. *IEEE Trans. Inf. Theory* **44**, 333–334 (1998).
34. R. L. Graham, N. J. A. Sloane, Lower bounds for constant weight codes. *IEEE Trans. Inf. Theory* **26**, 37–43 (1980).
35. J. H. Conway, N. J. A. Sloane, Lexicographic codes: Error-correcting codes from game theory. *IEEE Trans. Inf. Theory* **32**, 337–348 (1986).
36. G. Stern, H. Lenz, Steiner triple systems with given subspaces: Another proof of the Doyen-Wilson theorem. *Bollettino Unione Mat. Ital.* **17**, 109–114 (1980).
37. A. E. Brouwer, J. B. Shearer, N. J. A. Sloane, W. D. Smith, A new table of constant weight codes. *IEEE Trans. Inf. Theory* **36**, 1334–1380 (1990).
38. M. Braun, J. Humpich, A. Laaksonen, P. R. J. Östergård, New lower bounds on binary constant weight error-correcting codes. *J. Comb. Math. Comb. Comput.* **111**, 213–224 (2019).
39. Y. Lu, M. Liu, J. Yang, S. M. Weissman, X. Pan, S. G. Katz, S. Wang, Spatial transcriptome profiling by MERFISH reveals fetal liver hematopoietic stem cell niche architecture. *Cell Discov.*, 47 (2021).
40. M. E. Kastriti, L. Faure, D. Von Ahsen, T. G. Boudierlique, J. Boström, T. Solovieva, C. Jackson, M. Bronner, D. Meijer, S. Hadjab, F. Lallemand, A. Erickson, M. Kauka, V. Dyachuk, T. Perlmann, L. Lahti, J. Krivanek, J. Brunet, K. Fried, I. Adameyko, Schwann cell precursors represent a neural crest-like state with biased multipotency. *EMBO J.* **41**, e108780 (2022).
41. A. E. Brouwer, T. Etzion, A. E. Brouwer, T. Etzion, Some new distance-4 constant weight codes. *Adv. Math. Commun.* **5**, 417–424 (2011).
42. J. Bao, L. Ji, The completion determination of optimal (3,4)-packings. *Des. Codes Cryptogr.* **77**, 217–229 (2015).
43. L. Tian, F. Chen, E. Z. Macosko, The expanding vistas of spatial transcriptomics. *Nat. Biotechnol.* **41**, 773–782 (2023).
44. R. C. Team, R: A language and environment for statistical computing. R Foundation for Statistical. *Comput. Secur.* (2021). <https://www.r-project.org/>.
45. H. Wickham, R. Francois, dplyr: A Grammar of Data Manipulation. (2016). <https://cran.r-project.org/package=dplyr>.

Acknowledgments: We thank our colleagues in the Adameyko Laboratory for insightful discussions, and especially Y. Fatieieva for extracting the data used for simulating gene expressions in Fig. 4. **Funding:** This work was supported by: Paradifference Foundation (I.A.), Swedish Cancer Society (I.A.), Bertil Hållsten Research Foundation (I.A.), Knut and Alice Wallenberg Foundation project grant (I.A.), ERACoSysMed 4D-Healing grant (I.A.), Swedish Research Council (I.A.), ERC Consolidator Grant “STEMMING FROM NERVE,” 647844 (I.A.), ERC Synergy Grant “KILL OR DIFFERENTIATE,” 856529, ERC-2019-SyG (I.A.), Austrian Science Fund (FWF) (I.A.), EMBO Young Investigator (I.A.), Brain Resilience (I.A.), Emerging Fields-FWF-Brain Resilience (I.A.), and FWF Consortium Grant SFB F78 (I.A.). **Author contributions:** Conceptualization: J.B. and M.Z. Methodology: J.B. and M.Z. Investigation: J.B. Visualization: J.B. and I.A. Supervision: I.A. Writing—original draft: J.B., M.Z., and I.A. Writing—review and editing: J.B., M.Z., and I.A. **Competing interests:** The authors declare they have no competing interests. **Data and materials availability:** All data are available in the main text and/or the Supplementary Materials. All R codes are supplied in data S1 with documentation in the Supplementary Materials and also hosted on github.com/Adameykolab. Modified MATLAB code to generate large Extended Hamming Codes is supplied as data S2. All generated constant weight error-robust codebooks are supplied as data S3, S4, and S5, and are also deposited on DRYAD (DOI: 10.5061/dryad.zkh1893m5).

Submitted 2 July 2024

Accepted 28 March 2025

Published 2 May 2025

10.1126/sciadv.adr4026

Boosting multiplexing capabilities for error-robust spatial transcriptomic methods using a set exchange approach

Johan Boström, Micha# Zapa#a, and Igor Adameyko

Sci. Adv. **11** (18), eadr4026. DOI: 10.1126/sciadv.adr4026

View the article online

<https://www.science.org/doi/10.1126/sciadv.adr4026>

Permissions

<https://www.science.org/help/reprints-and-permissions>

Use of this article is subject to the [Terms of service](#)

Science Advances (ISSN 2375-2548) is published by the American Association for the Advancement of Science. 1200 New York Avenue NW, Washington, DC 20005. The title *Science Advances* is a registered trademark of AAAS.

Copyright © 2025 The Authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original U.S. Government Works. Distributed under a Creative Commons Attribution NonCommercial License 4.0 (CC BY-NC).